

```

ui->setupUi(this);
buf.setData(ar);
buf.open(QBuffer::ReadWrite);
connect(&buf, SIGNAL(readyRead()), this, SLOT(readData()));
}
Dialog::~Dialog() {
    delete ui;
}
void Dialog::on_pushButton_clicked() {
    buf.seek(0);
    buf.write(ui->lineEdit->text().toAscii().data());
}
void Dialog::readData() {
    buf.seek(0);
    ui->plainTextEdit->insertPlainText(buf.readAll());
}

```

In the above code, a *QBuffer* connected to a *QByteArray* has data from a *LineEdit* written to it when the pushbutton is clicked. In the constructor, the *readyRead()* signal is connected to the *readData()* slot, so when data is written to *buf*, the *readData()* slot will be called. The same thing happens in the hardware. So by doing a few tweaks in the code, you can emulate some hardware.

File dialogue

Now we move to graphical classes related to file operations. One of the most common is the File Open dialogue, which we have all seen. In Qt, this is done with the *QFileDialog* class, as in the following sample code:

```

void Dialog::on_pushButton_clicked() {
    QString str = QFileDialog::getOpenFileName(this, "Open File",
        ".", "Text File (*.txt)");
    if(!str.isEmpty()) {
        ui->lineEdit->setText(str);
        QFile file(str);
        if(file.open(QIODevice::ReadOnly | QIODevice::Text)) {
            ui->textEdit->clear();
            QString line;
            while(!file.atEnd()) {
                line = file.readLine();
                ui->textEdit->append(line);
            }
        }
    }
}

```

We invoked the *QFileDialog::getOpenFileName* call, which requires the path and the type of file to open as arguments. We specified the path as the present directory, and the file type as *.txt. The resultant dialogue can be seen in Figure 1. When the user selects a file and clicks *Open*, the function returns the path

and filename, which can be collected in a *QString*.

There are many operations we can perform on a file, such as truncate, append, delete, rename, etc. There is also directory handling, mostly similar to files, since a directory is itself a file. Qt provides excellent documentation on this; I suggest you read it, since I can't cover all of this.

The Qt resource system

So let's move on. Sometimes we may need to carry files along with the executable, such as icons. While it's easiest to copy the executable and other files in a single directory and ship it, Qt provides a better option. The Qt resource system provides a mechanism to store extra files within the executable itself. To use this:

1. Create a project. In the project file tree, click the project > *Add New > Qt Resource File*. A new file will be created in the *Resources* folder, with a 'qrc' extension.
2. Double-click the file to open it. You will find it empty.
3. Click *Add > Add Prefix* and name it in the *Prefix* field; for example, 'files'. You will find that in the resource file, a path 'files' has been created.
4. Select the path and navigate to *Add > Add Files* and add some files to it. I added the *main.cpp* file, as seen in Figure 3. That's it, you're done.

Accessing this file in your application is very easy:

```

#include <QtCore/QCoreApplication>
#include <QtCore>
int main(int argc, char *argv[]) {
    QCoreApplication a(argc, argv);
    QFile file;
    file.setFileName(":/files/main.cpp");
    if(!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
        qDebug() << "File not opened";
        return -1;
    }
    qDebug() << file.readAll();
    return a.exec();
}

```

Simply use the notation *':/prefix/filename'* to specify the resource file path that you want to open. The rest is easy to understand.

I hope this article helps you to work on files in your application. **END** 

By: Manoj Kumar

The author is a freelance developer and trainer. He leads a team in Linux kernel programming, Linux administration, cluster computing, embedded systems and Qt/GTK programming on Linux. You can reach him at manoj@nixitsolutions.com and read his blog, www.open4india.blogspot.com, to share your Linux knowledge.